

LLP TCA2 — Quick Reference Cheat Sheet

GCC Compilation Commands

Stage	Command	Output
Pre-process	<code>gcc -E Greeting.c -o Greeting.i</code>	<code>.i</code> file (expanded source)
Assemble to ASM	<code>gcc -S Greeting.c -o Greeting.s</code>	<code>.s</code> file (assembly code)
Compile to EXE	<code>gcc Greeting.c -o Greeting.exe</code>	<code>.exe</code> executable
Multiple files	<code>gcc main.c functions.c -o app.exe</code>	single executable

Run executable: `Greeting.exe` (or `./Greeting.exe` on Linux)

Struct — Define & Use

```
struct dog {
    char chipId[15];
    char name[30];
    char breed[30];
};

// Access
struct dog d;
d.chipId = ... // NOT valid for arrays – use scanf / strcpy
scanf("%14s", d.chipId);

// Via pointer
struct dog *p = &d;
printf("%s", p->name); // use -> with pointer
```

Dynamic List of STATIC Structs

```
#include <stdlib.h>
int size = 3;
struct dog *dogList = (struct dog*)malloc(size * sizeof(struct dog));
if (!dogList) { printf("malloc failed"); return -1; }

// Access
dogList[0].chipId ...
(dogList + 1)->name ...

// Free
free(dogList);
dogList = NULL;
```

Dynamic List of DYNAMIC Structs (pointer-to-pointer)

```
struct dog **dList = (struct dog**)malloc(10 * sizeof(struct dog));

// Set all to NULL first
for (int i = 0; i < 10; i++) dList[i] = NULL;

// Allocate each record
dList[0] = (struct dog*)malloc(sizeof(struct dog));

// Delete record at index 1
free(dList[1]);
dList[1] = NULL;

// Free everything
for (int i = 0; i < 10; i++) {
    if (dList[i]) { free(dList[i]); dList[i] = NULL; }
}
free(dList);
dList = NULL;
```

Binary File — Write & Read

```
// WRITE (wb)
FILE *fp = fopen("dogData.bin", "wb");
if (!fp) { printf("Error!"); return -1; }
fwrite(&dogList[i], sizeof(dogList[i]), 1, fp);
fclose(fp);

// READ (rb)
FILE *fp = fopen("dogData.bin", "rb");
if (!fp) { printf("Error!"); return -1; }
struct dog d;
int count = 0;
while (fread(&d, sizeof(d), 1, fp) == 1) {
    dogList[count++] = d;
}
fclose(fp);
```

Text File — Write & Read

```
// WRITE/APPEND
FILE *fp = fopen("log.txt", "a"); // "w" overwrites, "a" appends
fprintf(fp, "Hello %s\n", name);
fclose(fp);

// READ
FILE *fp = fopen("log.txt", "r");
char buffer[500];
```

```
while (fgets(buffer, sizeof(buffer), fp)) {
    printf("%s", buffer);
}
fclose(fp);
```

Header File (myHeader.h)

```
// myHeader.h
#ifndef MYHEADER_H
#define MYHEADER_H

struct dog {
    char chipId[15];
    char name[30];
    char breed[30];
};

struct dog fillRecord();
void displayRecord(struct dog d);

#endif
```

```
// In your .c file
#include "myHeader.h"
```

Functions — Key Syntax

```
// Pass by value
int add(int a, int b) { return a + b; }

// Pass by pointer (out parameter)
int toNumeric(char *s, int *number) {
    if (!s || !isNumeric(s) || strlen(s) > 10) return 0;
    *number = atoi(s); // dereference to SET value
    return 1;
}
// Call: toNumeric(str, &num); <-- pass ADDRESS with &
```

isNumeric Function

```
#include <ctype.h>
_Bool isNumeric(char *str) {
    if (!str || strlen(str) == 0) return 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i])) return 0;
    }
}
```

```
    return 1;
}
```

VS Code Debugging — Key Steps

1. **Set breakpoint** — click left gutter next to line number (red dot appears)
2. **Start debug** — press F5 or Run → Start Debugging
3. **Step Over** — F10 (execute line, don't enter function)
4. **Step Into** — F11 (enter into function)
5. **Watch variables** — Add variable name in Watch panel; values update live
6. **Hover** — hover over variable to see current value

Dynamic List vs Array — Key Advantages/Disadvantages

	Dynamic List of Static Structs	Array of Structs
Advantage	Size decided at runtime; no wasted memory	Simple syntax, no malloc needed
Disadvantage	Must use malloc/free; risk of memory leaks	Fixed size set at compile time
	Dynamic List of Dynamic Structs	Dynamic List of Static Structs
-	-----	-----
Advantage	Each record allocated individually; can delete without shifting	Simpler — one malloc for whole list
Disadvantage	More complex; must free each record + the list	Cannot delete individual records cleanly; freeing one record leaves gaps

Essential #include Headers

Header	Use for
<stdio.h>	printf, scanf, FILE, fopen, fclose, fread, fwrite, fprintf, fgets
<stdlib.h>	malloc, free, realloc, atoi, exit
<string.h>	strlen, strcpy, strcmp, strcat
<ctype.h>	isdigit, isalpha, isspace
"myHeader.h"	your own header (quotes, not angle brackets)